L16: Basic Backpropagation

C. V. Jawahar

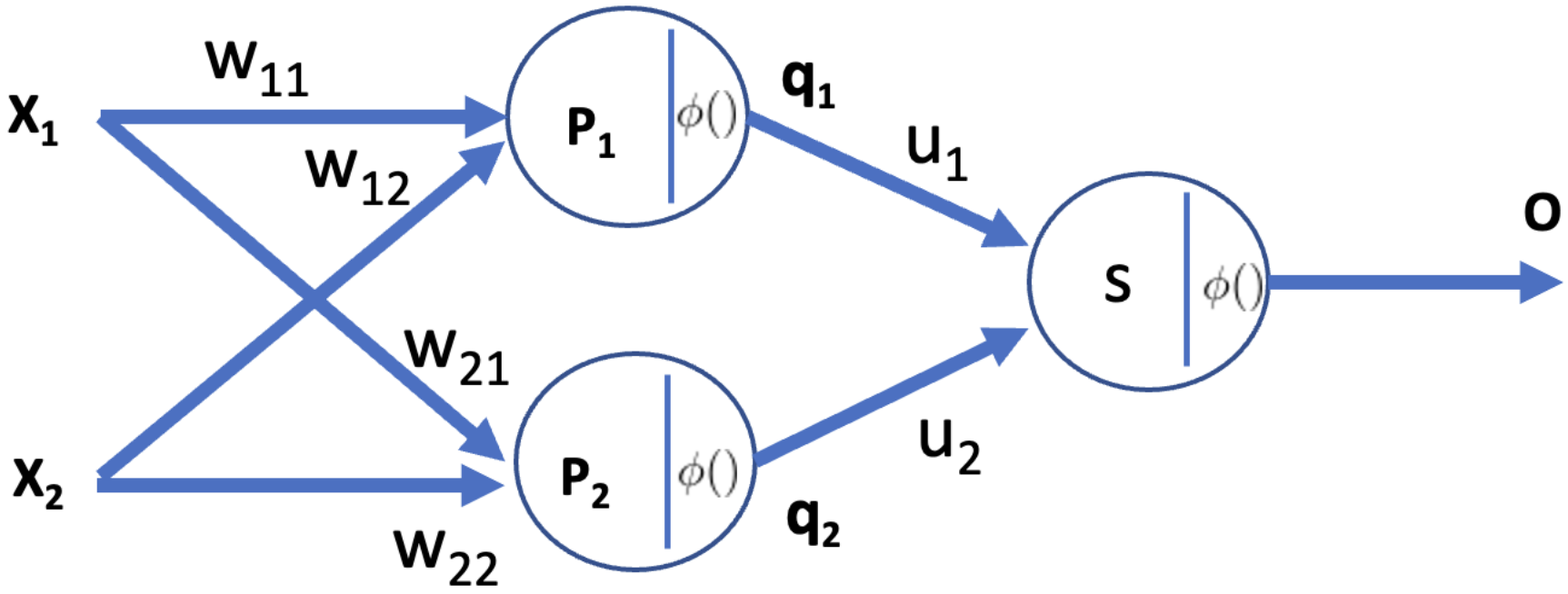
IIIT Hyderabad

March 12, 2026

Recap of Back Propagation

1. Initialize the network with random weights.
2. For all the samples, compute the output of the neural network $\mathbf{x}_{p+1}$ (or o)
$$\mathbf{x}_{m+1} = f_m(\mathbf{x}_m)$$
3. Compute the loss for the full batch of N samples as
$$L = \frac{1}{N}\sum_{i=1}^{N}\mathcal{L}(\mathbf{x}^i_{p+1}, \mathbf{y}_i)$$
4. Adjust all the parameters (such as weight matrices) as
$$\theta^{k+1} \leftarrow \theta^k - \Delta\theta$$
or
$$\theta^{k+1} \leftarrow \theta^k - \eta\frac{\partial L}{\partial \theta}$$
5. Repeat steps 2-4 until convergence.

A simple MLP

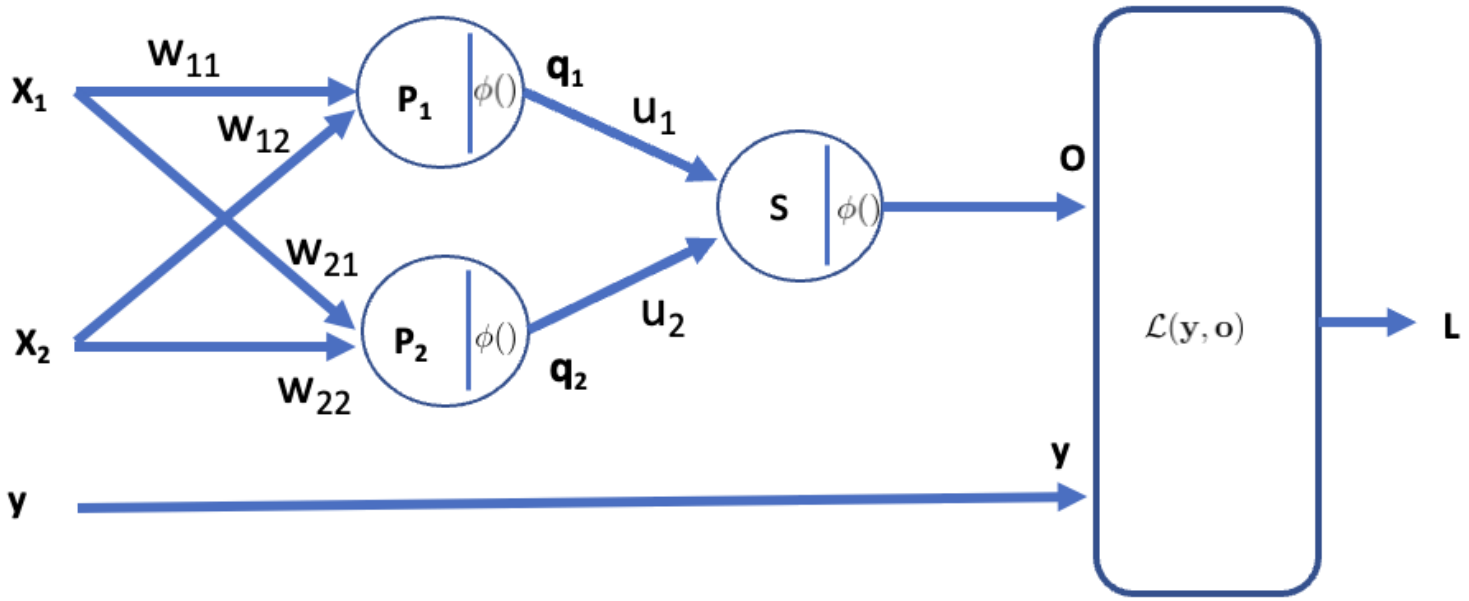


$$\begin{bmatrix} p_1 \\ p_2 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \; ; \; \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} = \begin{bmatrix} \phi(p_1) \\ \phi(p_2) \end{bmatrix}$$

$$s = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}^T \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} \; ; \; o = \phi(s)$$

$$\mathbf{p} = \mathbf{W}\mathbf{x} \; ; \; \mathbf{q} = \phi(\mathbf{p}) \; ; \; s = \mathbf{u}^T\mathbf{q} \; ; \; o = \phi(s)$$

MLP with Loss Layer

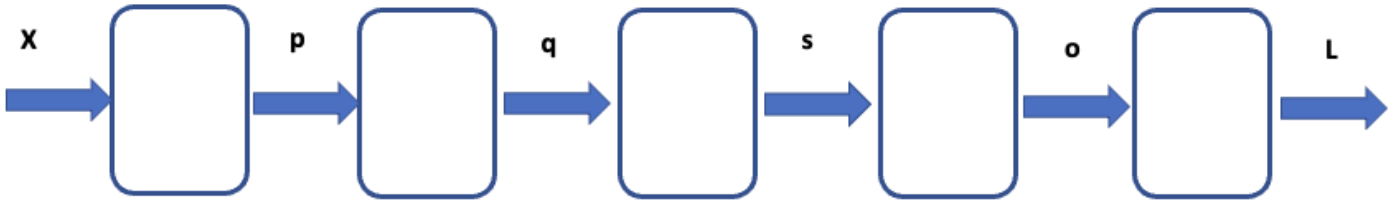


$$\begin{bmatrix} p_1 \\ p_2 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \; ; \; \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} = \begin{bmatrix} \phi(p_1) \\ \phi(p_2) \end{bmatrix}$$

$$s = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}^T \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} \; ; \; o = \phi(s) \; ; \; L = \mathcal{L}(y, o) = (o - y)^2$$

$$\mathbf{p} = \mathbf{W}\mathbf{x} \; ; \; \mathbf{q} = \phi(\mathbf{p}) \; ; \; s = \mathbf{u}^T\mathbf{q} \; ; \; o = \phi(s) \; ; \; L = (o - y)^2$$

Appreciating NNs as a Computational Graph



$$\mathbf{p} = \mathbf{W}\mathbf{x} \; ; \; \mathbf{q} = \phi(\mathbf{p}) \; ; \; s = \mathbf{u}^T\mathbf{q} \; ; \; o = \phi(s) \; ; \; L = (o - y)^2$$

- Some layers have learnable parameters; Some do not have any learnable parameters. No problem.
- Which layers in the above have learnable parameters?

Calculus: Primer

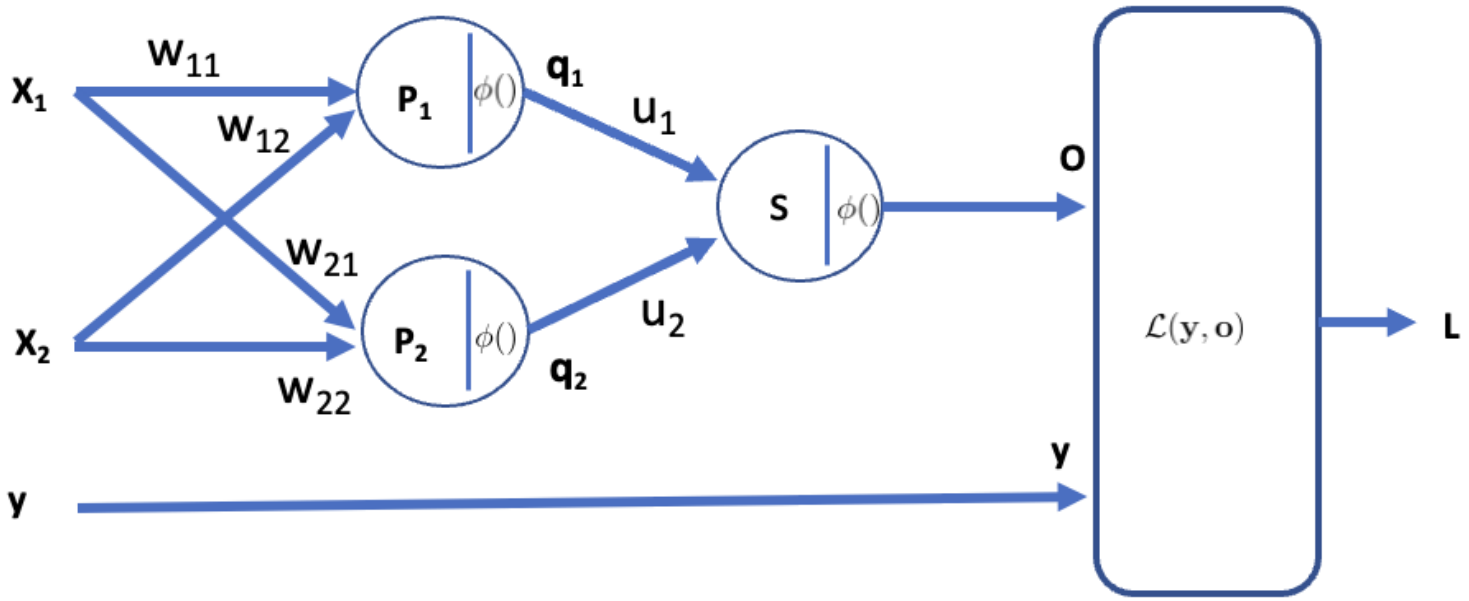
$$\frac{\partial(wx)}{\partial x} = w \ ; \ \frac{\partial(wx)}{\partial w} = x$$

$$\frac{\partial(o-y)^2}{\partial o} = 2(o-y) \ ;$$

$$\text{If } \phi(z) = \frac{1}{1+e^{-z}},$$

$$\frac{d\phi(z)}{dz} = \phi'(z) = \phi(z)(1-\phi(z))$$

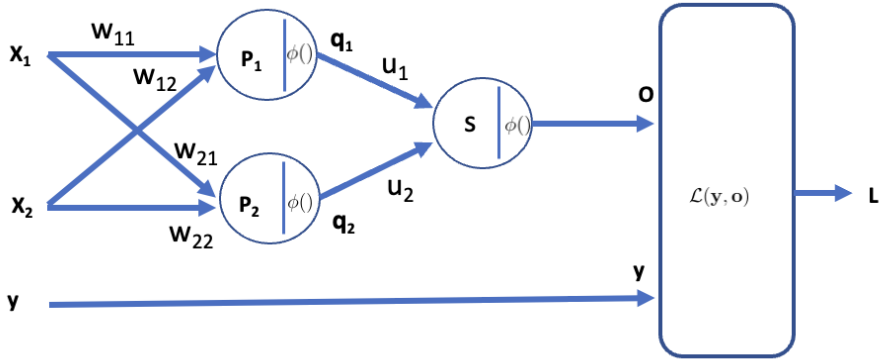
Computation of Derivatives: Chain Rule



$$\frac{\partial L}{\partial u_1} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial s} \cdot \frac{\partial s}{\partial u_1}$$
$$= 2(o - y) \cdot \phi'(s) \cdot q_1$$

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial s} \cdot \frac{\partial s}{\partial q_1} \cdot \frac{\partial q_1}{\partial p_1} \cdot \frac{\partial p_1}{\partial w_{11}}$$
$$= 2(o - y) \cdot \phi'(s) \cdot u_1 \cdot \phi'(p_1) \cdot x_1$$

Efficiency: Re-usability of computations in BP



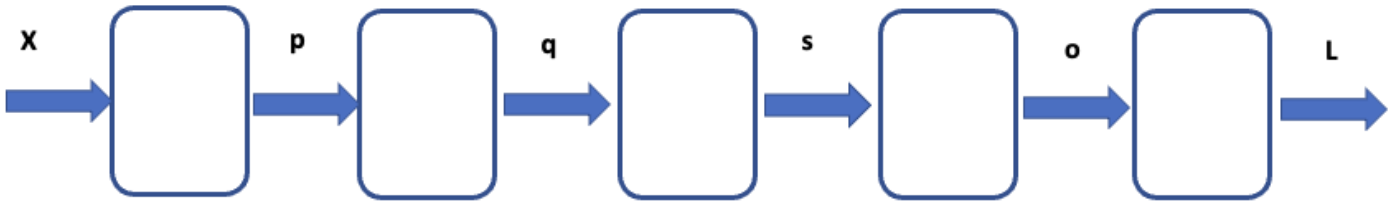
$$\frac{\partial L}{\partial u_1} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial s} \cdot \frac{\partial s}{\partial u_1}$$
$$= 2(o - y) \cdot \phi'(s) \cdot q_1$$

Reusable terms when we compute $\frac{\partial L}{\partial u_2}$

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial s} \cdot \frac{\partial s}{\partial q_1} \cdot \frac{\partial q_1}{\partial p_1} \cdot \frac{\partial p_1}{\partial w_{11}}$$
$$= 2(o - y) \cdot \phi'(s) \cdot u_1 \cdot \phi'(p_1) \cdot x_1$$

Reusable terms when we compute $\frac{\partial L}{\partial w_{12}}$

Backward Propagation of Error Gradients



$$\frac{\partial L}{\partial o} \tag{1}$$

$$\frac{\partial L}{\partial s} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial s} \tag{2}$$

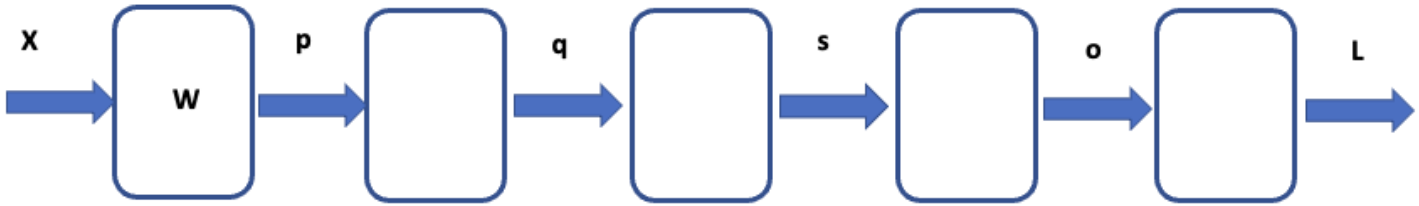
$$\frac{\partial L}{\partial q} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial s} \cdot \frac{\partial s}{\partial q} \tag{3}$$

$$\frac{\partial L}{\partial p} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial s} \cdot \frac{\partial s}{\partial q} \cdot \frac{\partial q}{\partial p} \tag{4}$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial s} \cdot \frac{\partial s}{\partial q} \cdot \frac{\partial q}{\partial p} \cdot \frac{\partial p}{\partial x} \tag{5}$$

Notice the re-usability of terms when computed backward.

Updating Weights (aka Learning)



$$\frac{\partial L}{\partial p} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial s} \cdot \frac{\partial s}{\partial q} \cdot \frac{\partial q}{\partial p}$$

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial W}$$

$$W^{K+1} \leftarrow W^{k} - \eta \frac{\partial L}{\partial W^{K}}$$

Backpropagation

- **Step 0:** Initialize the Network (MLP), weights, W^0.
- **Step 1:** Do forward pass for a mini batch (B) of randomly selected samples. Predict outputs with the existing weights.

$$o_i = f(\mathbf{W}, x_i)$$

- **Step 2:** Compute Loss for the set of samples.

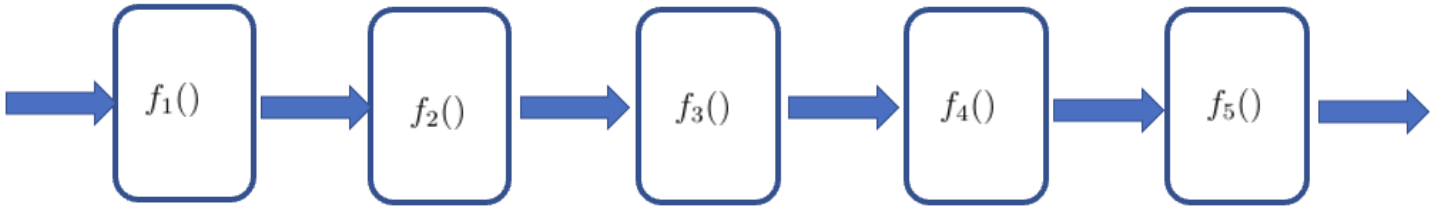
$$L = \frac{1}{|B|} \sum_{i \in B} \mathcal{L}(o_i, y_i)$$

- **Step 3:** Update all the weights using gradient descent.

$$W^{k+1} \leftarrow W^k - \eta \frac{\partial L}{\partial W^k}$$

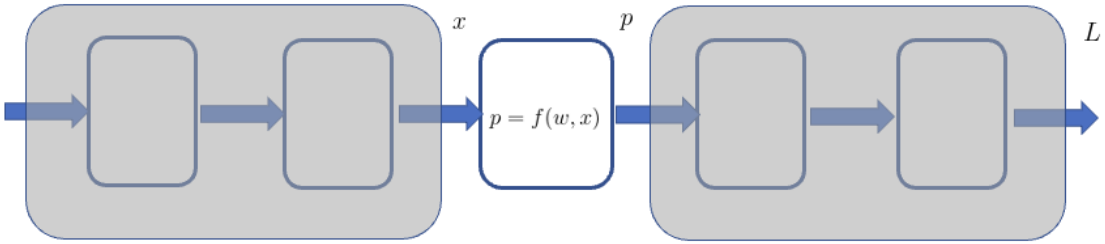
- **Step 4:** Repeat all steps (until the loss is acceptable.)

Notes



- If we compute gradients backward, it is efficient. "We propagate gradients backwards."
- Some blocks in the chain/graph may have learnable parameter and some may not have.
- Softmax, Activations, Loss etc. can be seen as layers (or some computational functions/blocks in the network).
- Long chain of multiplications can lead to "Vanishing gradients" or "exploding gradients".

Notes



- For every layer we use, we need to know how to compute derivative of output wrt input (i.e., $\frac{\partial p}{\partial x}$) so that gradients can be propagated backward.

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial x}$$

- For every layer, we need to know how to compute derivative of output wrt parameters/weights (i.e., $\frac{\partial p}{\partial w}$), (if we have) so that weights can be updated or we can train/learn.

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial w}$$

Problem (Advanced HW)

Define a custom "learnable sigmoid layer" in pytorch and train an MLP

$$\phi(w, z) = \frac{1}{1 + e^{-wz}}$$

Problem

1. Is it a good idea to initialize a NN with all weights as zero or equal? Why?

Problem

Plot derivative of Sigmoid. How does ReLU help in vanishing gradients?